

MLflow: A Beginner's Study Guide

Concepts, banking examples, practical workflow, and exercises | September 2026

1. The idea in one minute

Machine learning involves repeated attempts. You may train a decision tree, change its depth, try random forest, and then tune the probability threshold. Each attempt produces settings, scores, files, and decisions. MLflow is software for recording and comparing this work, saving model packages, and managing model versions.

Analogy: A scientist uses a lab notebook to record each experiment. MLflow is a searchable digital lab notebook for AI work. It does not train models by itself and does not guarantee a better model.

Banking story

Your team predicts loan default. Run A gives accuracy 88% and default recall 52%. Run B gives accuracy 85% and default recall 79%. If missing likely defaults is costly, Run B may be preferable despite its lower accuracy. MLflow shows the settings and results side by side; the bank still chooses the business objective.

Learning objectives

- Explain an experiment, run, parameter, metric, artifact, and logged model.
- Track at least two training attempts and compare meaningful test metrics.
- Load a saved model and understand how versioning and deployment fit into the workflow.

2. Why experiment tracking matters

A typical notebook can display a score and then lose the details when cells are rerun. A team may also have models saved under names such as `final.pkl`, `final2.pkl`, and `final_really_final.pkl`. Tracking ties each result to a particular attempt.

Question	What to record
What changed?	Algorithm, hyperparameters, preprocessing choice, threshold, code or data version.
How did it perform?	Test precision, recall, F1, ROC AUC, confusion matrix.
Can we reproduce it?	Random seed, dataset reference, environment, training settings.
Can we use it again?	Saved model with its preprocessing pipeline and input schema.

Core vocabulary

Term	Plain meaning	Example
Experiment	Group of related runs	Loan_Default
Run	One tracked attempt	Random forest, depth 5
Parameter	Chosen setting	<code>max_depth = 5</code>
Metric	Measured numeric result	<code>default_recall = 0.79</code>
Artifact	Stored output	<code>confusion_matrix.png</code>
Tag	Searchable label or note	<code>owner = banking_team</code>

Key distinction: Set a parameter before fitting; calculate a metric after evaluating. A model file is an artifact. A run ID identifies the recorded attempt. A registered model name is a separate managed identity for model versions.

Each term explained with a banking example

Experiment: A project folder for related model attempts. Create an experiment called `Loan_Default_2026`. It can contain attempts using logistic regression, decision trees, and random forests. An experiment is not a single trained model.

Run: One recorded attempt inside an experiment. Training a tree with `max_depth=3` is one run; training the same tree with `max_depth=6` is another run. Even if a run fails, its recorded information can help diagnose what happened.

Parameter: A setting chosen for the attempt, such as `max_depth=3`, `model_type=decision_tree`, or `test_size=0.20`. MLflow saves the value for comparison. Changing `max_depth` creates a meaningful new attempt.

Hyperparameter: A parameter that controls training, such as the number of trees. The model learns its internal split rules from data; you choose `n_estimators=100` before training. MLflow can log both training settings and descriptive parameters.

Metric: A numeric result you measure, such as `recall=0.79`. For loan default, recall tells you what fraction of actual defaulters were identified. Use names like `test_recall` so it is clear which split produced the number.

Artifact: A saved output, often a file: `confusion_matrix.png`, a CSV of predictions, or a trained model package. A metric is a number; an artifact can contain a chart or many files.

Tag: A searchable note attached to a run, for example `business_unit=retail_banking` or `reviewer=analyst`. Tags give context; never put secrets or customer records into them.

Model and model version: A logged model is the trained object plus information needed to load it. If a team registers selected models under `LoanDefaultClassifier`, each registration gets a version number. Run 12 and registered version 2 are different identifiers.

Tracking URI: The location where Python sends run information, such as `http://127.0.0.1:5000` for a local server. The MLflow UI is the browser view of that server.

3. What MLflow contains

Tracking and UI

Tracking receives run data. The UI lists experiments and lets you compare runs, inspect metrics, parameters, tags, and saved outputs. Your Python code sends information to a tracking URI. For local practice, this can be a server on your computer.

Backend store and artifact store

The **backend store** holds structured records such as experiment names, runs, parameters, and metrics. The **artifact store** holds larger output objects such as plots and model files. A local setup can keep both on one machine; a team setup can use a database and shared storage.

Models, registry, evaluation, serving

- **MLflow Model:** a packaged model with metadata and one or more ways to load or predict with it.
- **Model Registry:** a named collection of model versions. An alias can point to the version a team currently uses; aliases are easier to update than hard coded version numbers.
- **Evaluation:** compare candidate models using task appropriate metrics and additional reports. Metrics must be interpreted for the business question.
- **Serving:** make a packaged model available through a prediction endpoint. A local serving command is for demonstration; production deployment needs infrastructure and controls.
- **LLM/agent tracing:** inspect inputs, outputs, timing, retrieval steps, and tool calls in modern GenAI applications. Explore this after classical ML tracking.

MLflow Projects, dataset tracking, prompt management, and other features exist too. For a first introduction, focus on Tracking, Models, then Registry.

4. The workflow, from data to decision

Stage	Banking action	MLflow record
Define problem	Predict default; positive class = default	Experiment name and tags
Prepare data	Split training/test, encode categories	Data reference and preprocessing settings
Train candidates	Logistic regression, tree, forest	One run per meaningful attempt
Evaluate	Recall, precision, F1, ROC AUC	Metrics; confusion matrix artifact
Choose	Assess missed defaults and false alarms	Decision notes and run link
Reuse	Load model pipeline and predict	Logged model and optional registry version

Important evaluation discipline

- Keep the test set separate. Fit encoders and imputers only on training data using a pipeline.
- For imbalanced default labels, accuracy alone may hide missed defaulters. Log the positive class and its recall/precision.
- Keep one untouched test set for final selection. During extensive tuning, use training folds or validation data and avoid repeatedly optimizing against the test set.
- Record the dataset version or reliable reference. MLflow tracking does not automatically make a changed CSV reproducible.
- Log the complete preprocessing plus estimator pipeline, so new data receives the same transformations.

Three common misconceptions

“MLflow chooses the best model.” You define the decision criteria. **“High accuracy means deployment ready.”** You still need validation, fairness and operational checks. **“A registered version is live.”** Registry entries organize versions; a serving process must explicitly load or deploy one.

5. Set up a local demonstration

The following uses a local Python environment and MLflow server. Run the server and notebook on the same machine. In Jupyter, ensure the kernel uses the environment where packages are installed.

```
python -m pip install mlflow scikit-learn pandas
mlflow server
```

Leave that terminal open. Visit **http://127.0.0.1:5000**. In a separate terminal or notebook, connect and name the experiment:

```
import mlflow
mlflow.set_tracking_uri("http://127.0.0.1:5000")
mlflow.set_experiment("Banking_Loan_Default")
```

The tracking URI answers “where do my records go?” The experiment answers “which project should contain this run?” If a port is busy, start the server on another port and use the same address in `set_tracking_uri`.

Two simple ways to log

```
with mlflow.start_run(run_name="decision_tree_depth_3"):
    mlflow.log_param("max_depth", 3)
    mlflow.log_metric("test_recall", 0.72)
    mlflow.set_tag("positive_class", "default")
```

Start with manual logging to understand each item. Then try **`mlflow.sklearn.autolog()`** for automatic recording. Inspect the UI because autologging varies by estimator and does not replace deliberate test evaluation.

Explore results in the browser

Open Experiments; select `Banking_Loan_Default`; inspect the runs table; compare two runs; open a run to locate parameters, metrics, tags, and model outputs. Relate each result to the loan default question.

6. Practical example: two models

Use a banking CSV with its actual target column and a preprocessing pipeline. The following compact pattern shows exactly where MLflow belongs. The variables `X_train`, `X_test`, `y_train`, `y_test` and `preprocess` come from your ordinary notebook.

```
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import precision_score, recall_score, f1_score
import mlflow
import mlflow.sklearn

models = {
    "decision_tree": DecisionTreeClassifier(max_depth=4, random_state=42),
    "random_forest": RandomForestClassifier(
        n_estimators=100, max_depth=6, random_state=42)
}

for name, estimator in models.items():
    with mlflow.start_run(run_name=name):
        pipeline = Pipeline([
            ("preprocess", preprocess), ("model", estimator)
        ])
        pipeline.fit(X_train, y_train)
        predictions = pipeline.predict(X_test)
        mlflow.log_param("algorithm", name)
        mlflow.log_metric("test_precision", precision_score(
            y_test, predictions, pos_label=1, zero_division=0))
        mlflow.log_metric("test_recall", recall_score(
            y_test, predictions, pos_label=1, zero_division=0))
        mlflow.log_metric("test_f1", f1_score(
            y_test, predictions, pos_label=1, zero_division=0))
        mlflow.sklearn.log_model(
            sk_model=pipeline, name="loan_default_pipeline")
```

Before running: confirm default is encoded as 1. If it is a string label, change `pos_label` accordingly. Use a validation set for model selection when developing; treat a final test set as a one time assessment.

7. Saving, loading, and model versions

After logging, MLflow displays a model location within the run. You can load a model by its run-based URI or a registered model URI. The exact URI should be copied from the UI for the model you created.

```
import mlflow.sklearn

# Replace the placeholders with values copied from your MLflow UI.
model_uri = "runs:/RUN_ID/loan_default_pipeline"
loaded_pipeline = mlflow.sklearn.load_model(model_uri)
predictions = loaded_pipeline.predict(X_new)
```

A registered model is a named collection of versions. For example, `LoanDefaultClassifier` version 1 may point to a decision tree and version 2 to a random forest. You can assign a meaningful alias, such as `champion`, after your team approves a version. Registration is an organizational action; it does not automatically send predictions to users.

Version versus run

Run	Registered model version
Records one attempt and its evidence	References one packaged model under a managed name
Many runs may have no registered model	Only selected candidates usually need registration
Useful for experimentation	Useful for promoting, referencing, and managing candidates

Deployment and monitoring

Serving means accepting new input and returning predictions. Monitoring means checking what happens after deployment: input changes, errors, latency, and, once actual outcomes arrive, model quality. MLflow helps organize parts of this lifecycle, while operating a reliable banking service requires access controls, monitoring, review, and infrastructure beyond the introductory lab.

8. Practice exercises and review

Exercise 1: Identify the parts of a run

A bank trained a decision tree with `max_depth=4`. It recorded `test_recall=0.76` and saved `confusion_matrix.png`. State which item is a parameter, which is a metric, and which is an artifact. Then suggest a clear experiment name and run name.

Exercise 2: Compare two candidates

Candidate	Default precision	Default recall	Test F1
Decision tree	0.80	0.62	0.70
Random forest	0.72	0.84	0.78

Which model finds more actual defaulters? Which makes fewer false alarms among its positive predictions? Explain why the bank may prefer either model depending on the cost of a missed default versus an unnecessary manual review.

Exercise 3: Change one setting

Run a decision tree twice with `max_depth=3` and `max_depth=8`. Create separate MLflow runs, log the depth and test recall, and inspect them in the UI. Does the deeper tree always improve test results? Compare training and test scores to look for overfitting.

Exercise 4: Save and reuse

Log the complete preprocessing plus model pipeline. Copy its model URI from MLflow, load it, and predict on a small new table with the expected column names. Explain why saving the pipeline is useful when categorical variables require encoding.

Review questions

- How is an experiment different from a run?
- How does a parameter differ from a metric?
- Why can a model with lower accuracy be useful for loan default prediction?
- What information should be recorded to repeat a training attempt?
- What is the difference between logging a model, registering it, and serving it?

Common issues

- No runs in UI: check the server and tracking URI.
- Import error: install MLflow into the Python environment used by the notebook kernel.
- Different results each run: use a fixed `random_state` and inspect the split.
- Model cannot predict on new data: log the complete preprocessing pipeline and supply matching columns.
- Do not log passwords, API keys, or raw sensitive customer records.

Official references

Tracking: <https://mlflow.org/docs/latest/ml/tracking/> ; Quickstart: <https://mlflow.org/docs/latest/ml/tracking/quickstart/> ; Registry: <https://mlflow.org/docs/latest/ml/model-registry/> ; Server: <https://mlflow.org/docs/latest/self-hosting/architecture/tracking-server/> ; GenAI tracing: <https://mlflow.org/docs/latest/genai/tracing/>